

Investigar la definición de sentencias DML y aplicar al modelo de datos adjunto.

Juan José Guzman Bohorquez

Corporación Universitaria Del Huila

Neiva-Huila

2026

Definición de sentencias DML

Las sentencias DML (Data Manipulation Language) son instrucciones del lenguaje SQL que se utilizan para manipular los datos dentro de una base de datos. Es decir, permiten insertar, consultar, actualizar o eliminar información en las tablas.

Ejemplos:

- INSERT (agregar datos)
- SELECT (consultar datos)
- UPDATE (modificar datos)
- DELETE (eliminar datos)

Aplicación al modelo de datos

En el modelo dado, las sentencias DML se usan para gestionar la información de los esquemas **security**, **inventory** y **bill**.

Por ejemplo:

- INSERT para registrar usuarios, productos y facturas.
- SELECT para consultar la información guardada.
- UPDATE para modificar datos cuando sea necesario.
- DELETE para eliminar registros que ya no se usan.

Gracias a estas sentencias, el sistema puede funcionar correctamente y manejar toda la información.

Tabla: Role

```
CREATE TABLE security.role (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(100) NOT NULL,  
  description TEXT,  
  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  created_by VARCHAR(100),  
  updated_at TIMESTAMP,
```

```
updated_by VARCHAR(100),  
state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
```

Paso 1: Insertar

VALUES

```
('ADMIN', 'Administrador del sistema', 'system'),  
('USER', 'Usuario normal', 'system'),  
('SELLER', 'Vendedor', 'system');
```

Explicacion:

Aquí usamos el INSERT para agregar roles al sistema, como administrador o usuario.

Esto sirve para que cada persona que use la aplicación tenga un tipo de acceso diferente según su rol.

Paso 2: select

```
SELECT * FROM security.role;  
SELECT name, description FROM security.role;  
SELECT * FROM security.role WHERE state = 'ACTIVE';
```

Explicacion:

Con el SELECT lo que hacemos es ver los roles que ya están guardados en la base de datos.

Esto nos ayuda a revisar la información o mostrarla en el sistema cuando se necesite.

Paso 3:Delete

```
DELETE FROM security.role WHERE name = 'SELLER';  
DELETE FROM security.role WHERE name = 'USER';  
DELETE FROM security.role WHERE name = 'ADMIN';
```

Explicacion:

El DELETE se utiliza para eliminar roles que ya no se van a usar.

Así mantenemos la base de datos limpia y evitamos tener información innecesaria.

Tabla: User

```
CREATE TABLE security."user" (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  username VARCHAR(100) NOT NULL UNIQUE,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password TEXT NOT NULL,  
  role_id UUID NOT NULL,  
  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  created_by VARCHAR(100),
```

```
updated_at TIMESTAMP,  
updated_by VARCHAR(100),  
state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  
  
CONSTRAINT fk_user_role  
    FOREIGN KEY (role_id)  
    REFERENCES security.role(id)
```

Paso 1: Insert

```
INSERT INTO security."user" (username, email, password, role_id, created_by)  
VALUES  
( 'juan', 'juan@mail.com', '1234', (SELECT id FROM security.role WHERE  
name='ADMIN'), 'system'),  
( 'ana', 'ana@mail.com', '1234', (SELECT id FROM security.role WHERE  
name='USER'), 'system'),  
( 'carlos', 'carlos@mail.com', '1234', (SELECT id FROM security.role WHERE  
name='SELLER'), 'system');
```

Explicacion:

Aquí usamos el INSERT para registrar nuevos usuarios en el sistema, guardando datos como el nombre, correo, contraseña y el rol que tiene.
Esto permite que cada usuario pueda acceder a la aplicación con su propia cuenta.

Paso 2: Select

```
SELECT * FROM security."user";  
SELECT username, email FROM security."user";  
SELECT * FROM security."user" WHERE state = 'ACTIVE';
```

Explicacion:

Con el SELECT consultamos la información de los usuarios que ya están registrados.
Sirve para ver sus datos, validarlos o mostrarlos dentro del sistema cuando sea necesario.

Paso 3: delete

```
DELETE FROM security."user" WHERE username = 'juan';  
DELETE FROM security."user" WHERE username = 'ana';  
DELETE FROM security."user" WHERE username = 'carlos';
```

Explicacion:

El DELETE se utiliza para eliminar usuarios que ya no hacen parte del sistema.
De esta manera evitamos tener cuentas innecesarias y mantenemos la base de datos organizada.

Tabla:from

```
CREATE TABLE security.form (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  route VARCHAR(200),

  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  created_by VARCHAR(100),
  updated_at TIMESTAMP,
  updated_by VARCHAR(100),
  state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
```

Paso 1: insert

```
INSERT INTO security.form (name, route, created_by)
VALUES
('Login', '/login', 'system'),
('Dashboard', '/dashboard', 'system'),
('Usuarios', '/users', 'system');
```

Explicacion:

Aquí usamos INSERT para agregar formularios o pantallas del sistema, como login o dashboard.

Esto permite organizar las diferentes vistas que el usuario puede utilizar.

Paso 2: select

```
SELECT * FROM security.form;
SELECT name, route FROM security.form;
SELECT * FROM security.form WHERE state = 'ACTIVE';
```

Explicacion:

Con SELECT consultamos los formularios que existen en el sistema.

Sirve para ver qué pantallas están registradas o mostrarlas en la aplicación.

Paso 3:delete

```
DELETE FROM security.form WHERE name = 'Login';
DELETE FROM security.form WHERE name = 'Dashboard';
DELETE FROM security.form WHERE name = 'Usuarios';
```

Explicacion:

El DELETE se usa para eliminar formularios que ya no se utilizan.

Así evitamos tener opciones innecesarias dentro del sistema.

Tabla: categoría

```
CREATE TABLE inventory.category (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  description TEXT,
```

```
created_at TIMESTAMP NOT NULL DEFAULT NOW(),
created_by VARCHAR(100),
updated_at TIMESTAMP,
updated_by VARCHAR(100),
state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
```

Paso 1: Insert

```
INSERT INTO inventory.category (name, description, created_by)
VALUES
('Electrónica', 'Dispositivos electrónicos', 'system'),
('Ropa', 'Prendas de vestir', 'system'),
('Alimentos', 'Productos comestibles', 'system');
```

Explicacion:

Aquí usamos INSERT para crear categorías de productos, como electrónica o ropa. Esto ayuda a organizar los productos dentro del sistema.

Paso 2: select

```
SELECT * FROM inventory.category;
SELECT name FROM inventory.category;
SELECT * FROM inventory.category WHERE state = 'ACTIVE';
```

Explicacion:

Con SELECT podemos ver las categorías que ya están registradas. Sirve para consultar y mostrar la información cuando se necesite.

Paso 3: Delete

```
DELETE FROM inventory.category WHERE name = 'Electrónica';
DELETE FROM inventory.category WHERE name = 'Ropa';
DELETE FROM inventory.category WHERE name = 'Alimentos';
```

Explicacion:

El DELETE permite eliminar categorías que ya no se usan. Así mantenemos la base de datos ordenada.

Tabla:Product

```
CREATE TABLE inventory.product (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(150) NOT NULL,
  description TEXT,
  price NUMERIC(12,2) NOT NULL,
  category_id UUID NOT NULL,

  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  created_by VARCHAR(100),
```

```
updated_at TIMESTAMP,  
updated_by VARCHAR(100),  
state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  
  
CONSTRAINT fk_product_category  
FOREIGN KEY (category_id)  
REFERENCES inventory.category(id)
```

Paso 1:Insert

```
INSERT INTO inventory.product (name, description, price, category_id,  
created_by)  
VALUES  
('Laptop', 'Computador portátil', 2500, (SELECT id FROM inventory.category  
WHERE name='Electrónica'), 'system'),  
('Camiseta', 'Camiseta básica', 50, (SELECT id FROM inventory.category WHERE  
name='Ropa'), 'system'),  
('Pan', 'Pan integral', 5, (SELECT id FROM inventory.category WHERE  
name='Alimentos'), 'system');
```

Explicacion:

El DELETE se usa para eliminar formularios que ya no se utilizan.
Así evitamos tener opciones innecesarias dentro del sistema.

Paso 2:Select

```
SELECT * FROM inventory.product;  
SELECT name, price FROM inventory.product;  
SELECT * FROM inventory.product WHERE price > 100;
```

Explicacion:

Aquí usamos INSERT para agregar productos con su nombre, precio y categoría.
Esto permite registrar los productos que se van a vender o manejar en el sistema.

Paso 3:Delete

```
DELETE FROM inventory.product WHERE name = 'Laptop';  
DELETE FROM inventory.product WHERE name = 'Camiseta';  
DELETE FROM inventory.product WHERE name = 'Pan';
```

Explicacion:

El DELETE se usa para eliminar productos que ya no están disponibles.
Así evitamos mostrar productos que no existen.

Tabla:Inventory

```
CREATE TABLE inventory.inventory (  
  
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
product_id UUID NOT NULL,
```

```

quantity INTEGER NOT NULL,

created_at TIMESTAMP NOT NULL DEFAULT NOW(),
created_by VARCHAR(100),
updated_at TIMESTAMP,
updated_by VARCHAR(100),
state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',

CONSTRAINT fk_inventory_product
    FOREIGN KEY (product_id)
    REFERENCES inventory.product(id)
);

```

Paso 1: Insert

```

INSERT INTO inventory.inventory (product_id, quantity, created_by)
VALUES
((SELECT id FROM inventory.product WHERE name='Laptop'), 10, 'system'),
((SELECT id FROM inventory.product WHERE name='Camiseta'), 50, 'system'),
((SELECT id FROM inventory.product WHERE name='Pan'), 100, 'system');

```

Explicacion:

Aquí usamos INSERT para registrar la cantidad de productos disponibles. Esto permite llevar el control del stock dentro del sistema.

Paso 2 : select

```

SELECT * FROM inventory.inventory;
SELECT product_id, quantity FROM inventory.inventory;
SELECT * FROM inventory.inventory WHERE quantity > 20;

```

Explicacion:

Con SELECT vemos cuántos productos hay en inventario. Sirve para controlar existencias y tomar decisiones.

Paso 3: Delete

```

DELETE FROM inventory.inventory WHERE quantity = 10;
DELETE FROM inventory.inventory WHERE quantity = 50;
DELETE FROM inventory.inventory WHERE quantity = 100;

```

Explicacion:

El DELETE se utiliza para eliminar registros de inventario. Esto puede servir cuando ya no se necesita ese control o se va a actualizar.

Tabla: Bill


```
CREATE TABLE bill.bill (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL,
  total NUMERIC(14,2) NOT NULL,

  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  created_by VARCHAR(100),
  updated_at TIMESTAMP,
  updated_by VARCHAR(100),
  state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',

  CONSTRAINT fk_bill_user
    FOREIGN KEY (user_id)
    REFERENCES security."user"(id)
```

Paso 1:Insert

```
INSERT INTO bill.bill (user_id, total, created_by)
VALUES
((SELECT id FROM security."user" WHERE username='juan'), 3000, 'system'),
((SELECT id FROM security."user" WHERE username='ana'), 200, 'system'),
((SELECT id FROM security."user" WHERE username='carlos'), 150, 'system');
```

Explicacion:

Aquí usamos INSERT para crear facturas asociadas a un usuario.
Esto permite registrar las compras realizadas dentro del sistema.

Paso 2: Select

```
SELECT * FROM bill.bill;
SELECT user_id, total FROM bill.bill;
SELECT * FROM bill.bill WHERE total > 500;
```

Explicacion:

Con SELECT consultamos las facturas guardadas.
Sirve para ver el historial de compras o revisar información.

Paso 3:Delete

```
DELETE FROM bill.bill WHERE total = 3000;
DELETE FROM bill.bill WHERE total = 200;
DELETE FROM bill.bill WHERE total = 150;
```

Explicacion:

El DELETE se usa para eliminar facturas cuando es necesario.
Esto ayuda a mantener la base de datos limpia.

Tabla:Bill_item

```
CREATE TABLE bill.bill_item (
```

```

id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
bill_id UUID NOT NULL,
product_id UUID NOT NULL,
quantity INTEGER NOT NULL,
unit_price NUMERIC(12,2) NOT NULL,
total NUMERIC(14,2) NOT NULL,

created_at TIMESTAMP NOT NULL DEFAULT NOW(),
created_by VARCHAR(100),
updated_at TIMESTAMP,
updated_by VARCHAR(100),
state VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',

CONSTRAINT fk_bill_item_bill
    FOREIGN KEY (bill_id)
    REFERENCES bill.bill(id),

CONSTRAINT fk_bill_item_product
    FOREIGN KEY (product_id)
    REFERENCES inventory.product(id)

```

Paso 1: Insert

```

INSERT INTO bill.bill_item (bill_id, product_id, quantity, unit_price, total,
created_by)
VALUES
((SELECT id FROM bill.bill LIMIT 1),(SELECT id FROM inventory.product WHERE
name='Laptop'),1,2500,2500,'system'),
((SELECT id FROM bill.bill LIMIT 1),(SELECT id FROM inventory.product WHERE
name='Camiseta'),2,50,100,'system'),
((SELECT id FROM bill.bill LIMIT 1),(SELECT id FROM inventory.product WHERE
name='Pan'),10,5,50,'system');

```

Explicacion:

Aquí usamos INSERT para agregar los productos que tiene una factura. Esto permite detallar qué compró el usuario y en qué cantidad.

Paso 2: select

```

SELECT * FROM bill.bill_item;
SELECT quantity, total FROM bill.bill_item;
SELECT * FROM bill.bill_item WHERE total > 100;

```

Explicacion

Con SELECT consultamos los productos dentro de cada factura. Sirve para ver el detalle de las compras.

Paso 3: Delete

```
DELETE FROM bill.bill_item WHERE total = 2500;  
DELETE FROM bill.bill_item WHERE total = 100;  
DELETE FROM bill.bill_item WHERE total = 50;
```

Explicacion:

El DELETE se utiliza para eliminar productos de una factura.

Esto puede servir si hubo un error o se necesita corregir la información.